

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación paralela y distribuida - 40

Marlon Fuentes



Laboratorio 3

Ihan Marroquin - 23108

GUATEMALA, 03 de octubre del 2026

Informe final

Repositorio: https://github.com/Than-Marroquin/Labo3_paralela

Entorno y pruebas

Los cuatro programas se hicieron en C con Open MPI y se probaron en WSL, siempre en la misma computadora; para medir se usó **MPI_Wtime** y cada configuración se repitió siete veces, así las tablas no dependen de una sola corrida. Además del tiempo se guardaron datos como el valor recibido, el token final, el checksum y el mensaje de parada, porque también era necesario revisar que cada programa terminara bien.

Elemento	Configuración
Sistema	Ubuntu 26.04.1 LTS en WSL
Procesador	Intel Core i7-11800H, 8 núcleos y 16 hilos lógicos
MPI	Open MPI 5.0.10
Compilador	GCC 15.2.0 con -O2
Muestreo	7 repeticiones por tamaño

Parte 1 Comunicación con bloqueo

Ping Pong

Inciso 1 Implementación

El programa se ejecuta con dos procesos: el rank 0 llena el mensaje con el valor 42, lo manda al rank 1 y espera que regrese, mientras que el rank 1 solamente lo recibe y lo devuelve. Desde la terminal se puede cambiar tanto el número de iteraciones como la cantidad de enteros que lleva el mensaje.

```

60 }
61
62 MPI_Barrier(MPI_COMM_WORLD);
63 const double start = MPI_Wtime();
64
65 for (int iteration = 0; iteration < iterations; ++iteration) {
66     if (rank == 0) {
67         MPI_Send(message, count, MPI_INT, 1, TAG_PING, MPI_COMM_WORLD);
68         MPI_Recv(message, count, MPI_INT, 1, TAG_PONG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
69         if (detailed) {
70             printf("[rank 0] iteracion %d: ping enviado y pong recibido, valor=%d\n",
71                 iteration + 1, message[0]);
72             fflush(stdout);
73         }
74     } else {
75         MPI_Recv(message, count, MPI_INT, 0, TAG_PING, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
76         MPI_Send(message, count, MPI_INT, 0, TAG_PONG, MPI_COMM_WORLD);
77     }
78 }
79
80 MPI_Barrier(MPI_COMM_WORLD);
81 const double elapsed = MPI_Wtime() - start;
82

```

Figura 1. Intercambio bloqueante y medición con `MPI_Wtime` en `ping_pong.c`.

```

ihan@VICTUS:~/Labo3_paralela$ mpirun -np 2 ./build/ping_pong 5 16 --detalle
[rank 0] iteracion 1: ping enviado y pong recibido, valor=42
[rank 0] iteracion 2: ping enviado y pong recibido, valor=42
[rank 0] iteracion 3: ping enviado y pong recibido, valor=42
[rank 0] iteracion 4: ping enviado y pong recibido, valor=42
[rank 0] iteracion 5: ping enviado y pong recibido, valor=42
PING_PONG iteraciones=5 enteros=16 bytes=64 tiempo_total_s=0.000051952 tiempo_promedio_ms=0.010390400
valor_final=42

```

Figura 2. Cinco intercambios correctos con un mensaje de 16 enteros.

En la salida se ven las cinco respuestas con el valor 42, y para esta prueba el tiempo promedio de ida y vuelta fue de 0.010390 ms.

Inciso 2 Tamaño del mensaje y tiempo

Mensaje bytes	Promedio ms	Desviación ms
4	0.000322	0.000022
64	0.000358	0.000022
1,024	0.000968	0.000132
16,384	0.006163	0.000105
262,144	0.042520	0.001769

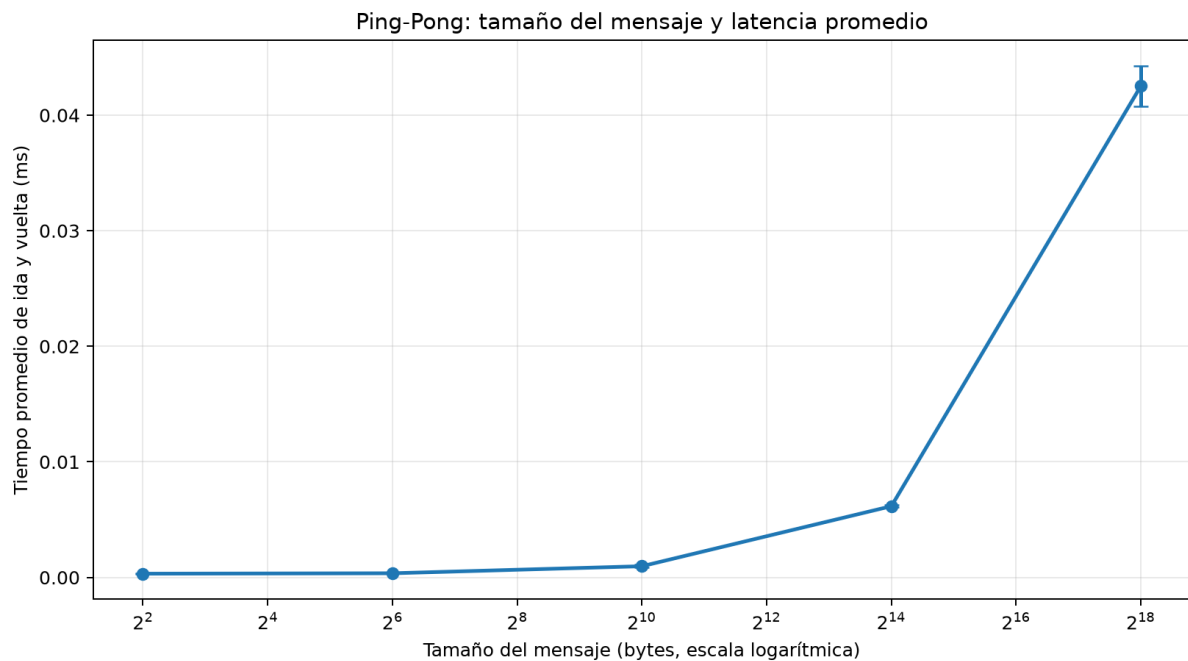


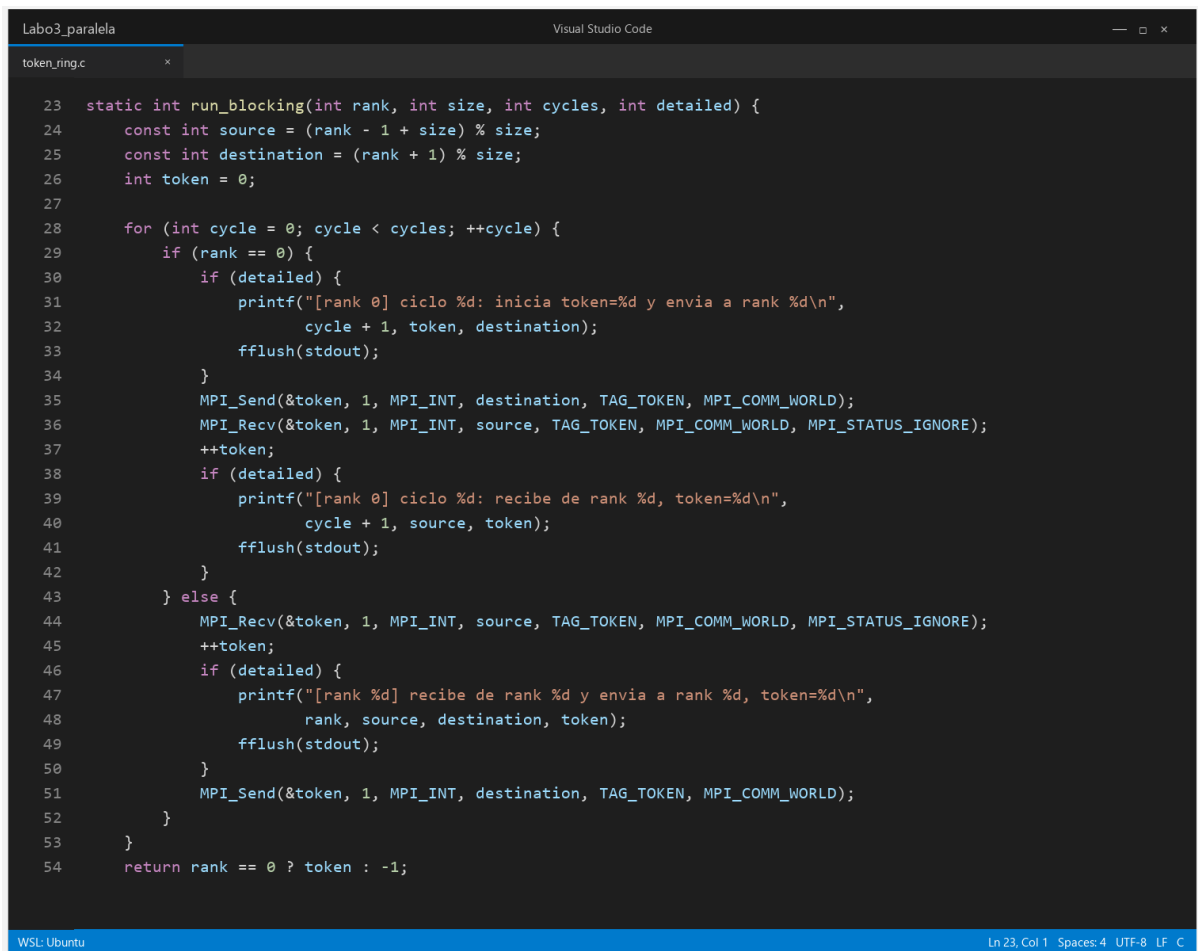
Figura 3. Tiempo promedio de ida y vuelta según el tamaño del mensaje. Las barras representan una desviación estándar.

Sí se nota una tendencia creciente, aunque entre 4 y 64 bytes casi no cambia el resultado, pues los promedios fueron de 0.000322 y 0.000358 ms y en ese punto pesa más iniciar la comunicación que mover los datos. Desde 1,024 bytes el aumento ya se ve con más claridad, hasta llegar a 0.042520 ms con 262,144 bytes, cerca de 132 veces el valor obtenido con 4 bytes.

Token Ring

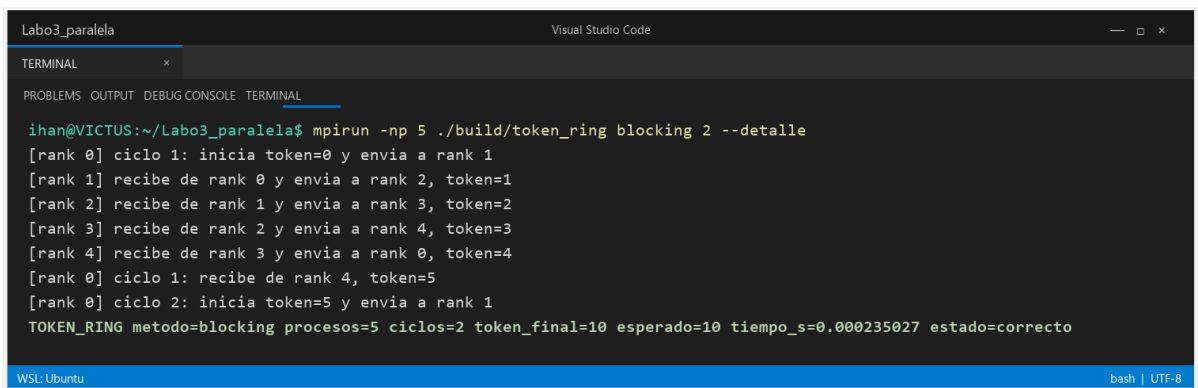
Inciso 1 Comunicación con MPI Send y MPI Recv

Cada proceso sabe de quién recibe y a quién envía usando las fórmulas **(rank - 1 + size) % size** y **(rank + 1) % size**. Para que el anillo no se quede trabado, el rank 0 empieza enviando el token y luego espera su regreso, mientras que los demás primero reciben y después lo pasan al siguiente proceso.



```
23 static int run_blocking(int rank, int size, int cycles, int detailed) {
24     const int source = (rank - 1 + size) % size;
25     const int destination = (rank + 1) % size;
26     int token = 0;
27
28     for (int cycle = 0; cycle < cycles; ++cycle) {
29         if (rank == 0) {
30             if (detailed) {
31                 printf("[rank 0] ciclo %d: inicia token=%d y envia a rank %d\n",
32                     cycle + 1, token, destination);
33                 fflush(stdout);
34             }
35             MPI_Send(&token, 1, MPI_INT, destination, TAG_TOKEN, MPI_COMM_WORLD);
36             MPI_Recv(&token, 1, MPI_INT, source, TAG_TOKEN, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
37             ++token;
38             if (detailed) {
39                 printf("[rank 0] ciclo %d: recibe de rank %d, token=%d\n",
40                     cycle + 1, source, token);
41                 fflush(stdout);
42             }
43         } else {
44             MPI_Recv(&token, 1, MPI_INT, source, TAG_TOKEN, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
45             ++token;
46             if (detailed) {
47                 printf("[rank %d] recibe de rank %d y envia a rank %d, token=%d\n",
48                     rank, source, destination, token);
49                 fflush(stdout);
50             }
51             MPI_Send(&token, 1, MPI_INT, destination, TAG_TOKEN, MPI_COMM_WORLD);
52         }
53     }
54     return rank == 0 ? token : -1;
}
```

Figura 4. Recorrido del token con MPI_Send y MPI_Recv.



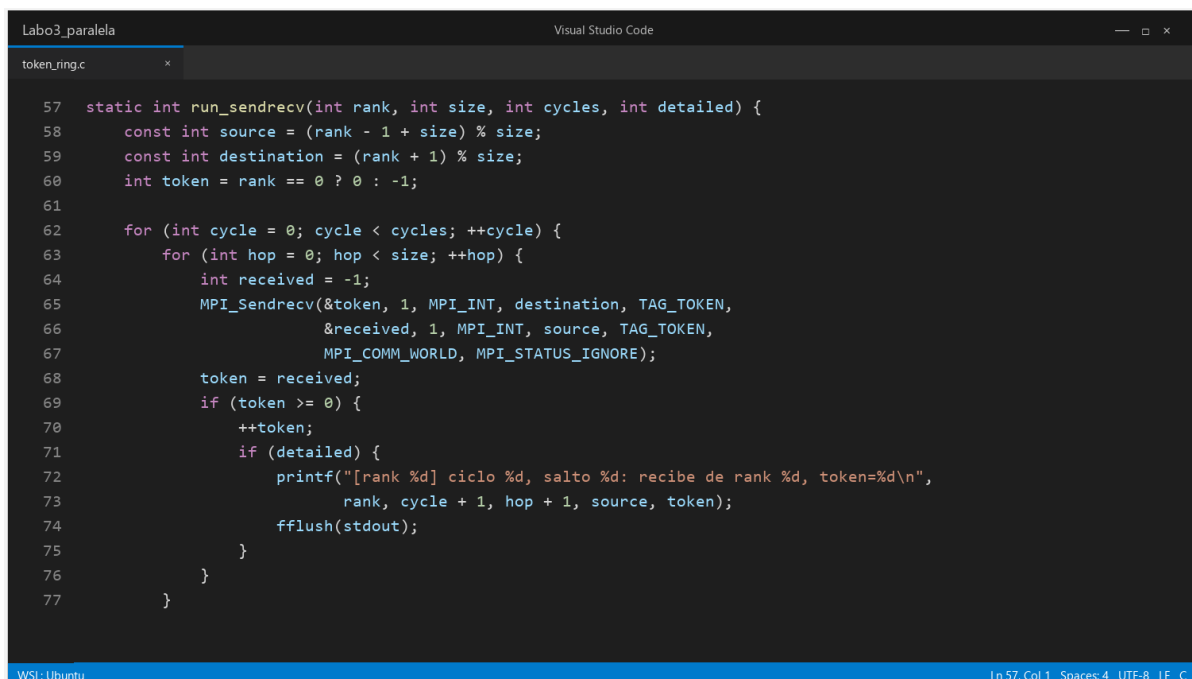
```
ihan@VICTUS:~/Labo3_paralela$ mpirun -np 5 ./build/token_ring blocking 2 --detalle
[rank 0] ciclo 1: inicia token=0 y envia a rank 1
[rank 1] recibe de rank 0 y envia a rank 2, token=1
[rank 2] recibe de rank 1 y envia a rank 3, token=2
[rank 3] recibe de rank 2 y envia a rank 4, token=3
[rank 4] recibe de rank 3 y envia a rank 0, token=4
[rank 0] ciclo 1: recibe de rank 4, token=5
[rank 0] ciclo 2: inicia token=5 y envia a rank 1
TOKEN_RING metodo=blocking procesos=5 ciclos=2 token_final=10 esperado=10 tiempo_s=0.000235027 estado=correcto
```

Figura 5. Primer ciclo del anillo bloqueante con cinco procesos.

Con cinco procesos y dos ciclos, el token terminó en 10 como se esperaba después de diez pasos, y la salida también deja ver que el rank 4 vuelve a enviar al rank 0 para cerrar el anillo.

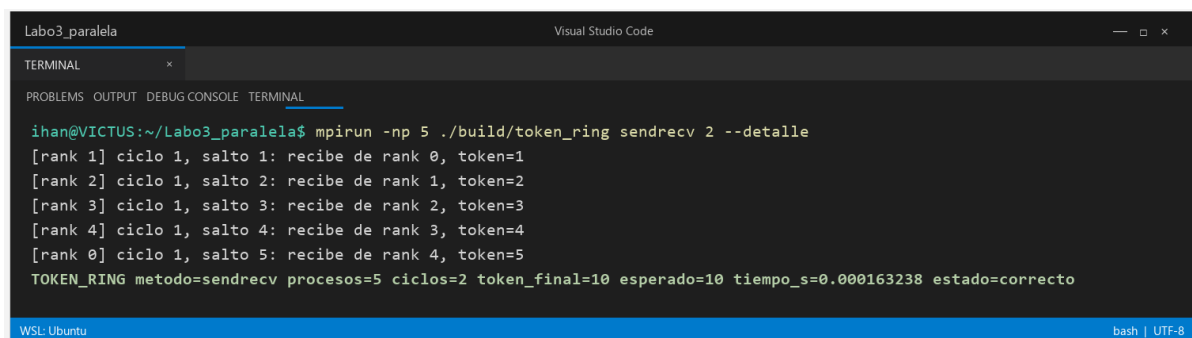
Inciso 2 Comunicación con MPI Sendrecv

En la segunda versión se usa una sola llamada para mandar el token al siguiente proceso y recibirlo del anterior, por eso solamente el rank 0 comienza con un valor válido y los demás arrancan con -1. Después de cinco saltos el token regresa al rank 0 y ya puede comenzar el siguiente ciclo.



```
57 static int run_sendrecv(int rank, int size, int cycles, int detailed) {
58     const int source = (rank - 1 + size) % size;
59     const int destination = (rank + 1) % size;
60     int token = rank == 0 ? 0 : -1;
61
62     for (int cycle = 0; cycle < cycles; ++cycle) {
63         for (int hop = 0; hop < size; ++hop) {
64             int received = -1;
65             MPI_Sendrecv(&token, 1, MPI_INT, destination, TAG_TOKEN,
66                         &received, 1, MPI_INT, source, TAG_TOKEN,
67                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
68             token = received;
69             if (token >= 0) {
70                 ++token;
71                 if (detailed) {
72                     printf("[rank %d] ciclo %d, salto %d: recibe de rank %d, token=%d\n",
73                             rank, cycle + 1, hop + 1, source, token);
74                     fflush(stdout);
75                 }
76             }
77         }
78     }
79 }
```

Figura 6. Paso del token mediante `MPI_Sendrecv`.



```
ih@VICTUS:~/Labo3_paralela$ mpirun -np 5 ./build/token_ring sendrecv 2 --detalle
[rank 1] ciclo 1, salto 1: recibe de rank 0, token=1
[rank 2] ciclo 1, salto 2: recibe de rank 1, token=2
[rank 3] ciclo 1, salto 3: recibe de rank 2, token=3
[rank 4] ciclo 1, salto 4: recibe de rank 3, token=4
[rank 0] ciclo 1, salto 5: recibe de rank 4, token=5
TOKEN_RING metodo=sendrecv procesos=5 ciclos=2 token_final=10 esperado=10 tiempo_s=0.000163238 estado=correcto
```

Figura 7. Un ciclo completo con `MPI_Sendrecv` y cinco procesos.

Inciso 3 Diferencia entre `Sendrecv` y `Send` más `Recv`

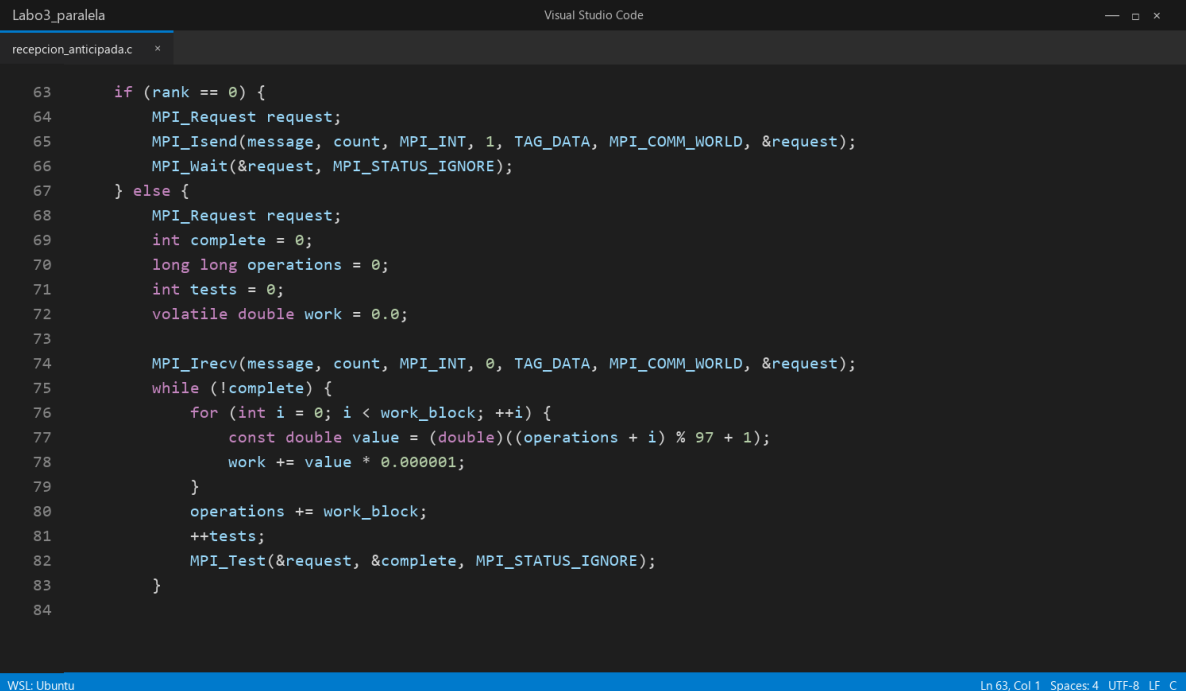
`MPI_Sendrecv` hace el envío y la recepción dentro de la misma llamada, por eso queda muy bien para un anillo, donde cada proceso debe mandar al siguiente y recibir del anterior. Con `MPI_Send` y `MPI_Recv` las dos acciones van por separado y hay que cuidar el orden para que todos no se queden esperando al mismo tiempo; en esta versión se resolvió haciendo que el rank 0 envíe primero y que los demás comiencen recibiendo. `Sendrecv` deja esa parte más directa, aunque la llamada continúa siendo bloqueante.

Parte 2 Comunicación sin bloqueo

Recepción anticipada

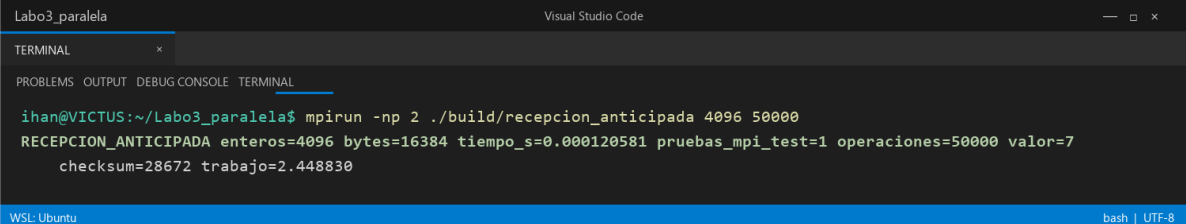
Incisos 1 y 2 Recepción con trabajo simultáneo

El rank 0 comienza el envío con **MPI_Isend** y el rank 1 deja preparada la recepción con **MPI_Irecv**; mientras llega el mensaje, aprovecha para hacer bloques de 50,000 operaciones y al final de cada bloque revisa con **MPI_Test** si ya terminó. Cuando la recepción se completa, imprime el primer valor, el checksum y la cantidad de trabajo realizado.



```
63     if (rank == 0) {
64         MPI_Request request;
65         MPI_Isend(message, count, MPI_INT, 1, TAG_DATA, MPI_COMM_WORLD, &request);
66         MPI_Wait(&request, MPI_STATUS_IGNORE);
67     } else {
68         MPI_Request request;
69         int complete = 0;
70         long long operations = 0;
71         int tests = 0;
72         volatile double work = 0.0;
73
74         MPI_Irecv(message, count, MPI_INT, 0, TAG_DATA, MPI_COMM_WORLD, &request);
75         while (!complete) {
76             for (int i = 0; i < work_block; ++i) {
77                 const double value = (double)((operations + i) % 97 + 1);
78                 work += value * 0.000001;
79             }
80             operations += work_block;
81             ++tests;
82             MPI_Test(&request, &complete, MPI_STATUS_IGNORE);
83         }
84     }
```

Figura 8. Recepción anticipada y consulta periódica con **MPI_Test**.



```
ihana@VICTUS:~/Labo3_paralela$ mpirun -np 2 ./build/repcion_anticipada 4096 50000
RECEPCION_ANTICIPADA enteros=4096 bytes=16384 tiempo_s=0.000120581 pruebas_mpi_test=1 operaciones=50000 valor=7
checksum=28672 trabajo=2.448830
```

Figura 9. Recepción de 4,096 enteros mientras el rank 1 realiza trabajo aritmético.

En la prueba con 4,096 enteros se conservó el valor 7 y el checksum fue de 28,672, además la recepción terminó después de una consulta a **MPI_Test** mientras el rank 1 alcanzó a hacer 50,000 operaciones.

Inciso 3 Tamaño del mensaje y tiempo

Mensaje bytes	Promedio ms	Desviación ms
4	0.115536	0.008507
4,096	0.118341	0.002999
65,536	0.128701	0.005979
1,048,576	0.345452	0.034372
4,194,304	1.079625	0.050387

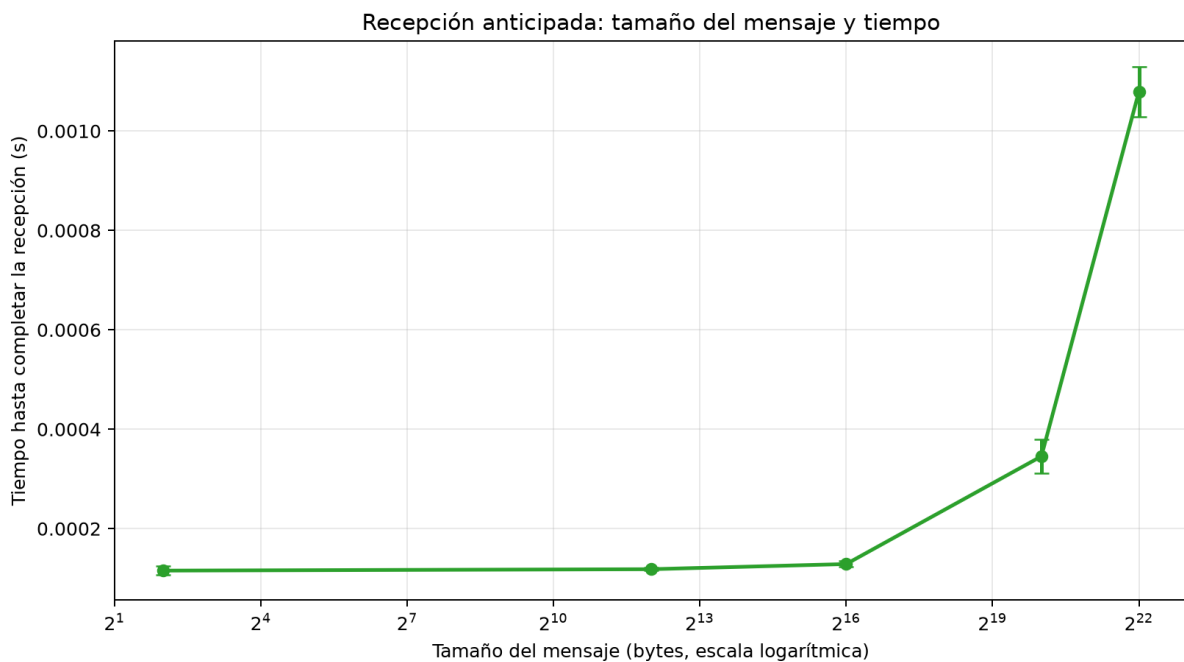


Figura 10. Tiempo hasta completar la recepción según el tamaño del mensaje.

Entre 4 y 65,536 bytes el tiempo se mantuvo bastante parecido, de 0.115536 a 0.128701 ms, porque en mensajes pequeños pesan más el bloque de trabajo y la consulta a MPI_Test. Con 1,048,576 bytes el promedio subió a 0.345452 ms y con 4,194,304 bytes llegó a 1.079625 ms, pero durante esa espera el rank 1 puede seguir avanzando con el cálculo.

Producer Consumer Pipeline por chunks

Inciso 1 Envío y procesamiento por chunks

El rank 0 crea un arreglo con valores entre 0 y 99 y puede mantener dos envíos activos al mismo tiempo. Del otro lado, el rank 1 prepara dos recepciones y usa **MPI_Waitany** para saber cuál buffer terminó primero; procesa ese chunk y vuelve a usar el mismo espacio, así puede ir recibiendo el siguiente bloque mientras trabaja con el anterior.

```

Labo3_paralela Visual Studio Code
pipeline_chunks.c x
94     if (rank == 0) {
95         MPI_Request sends[BUFFER_COUNT] = {MPI_REQUEST_NULL, MPI_REQUEST_NULL};
96         for (int chunk = 0; chunk < chunks; ++chunk) {
97             const int slot = chunk % BUFFER_COUNT;
98             if (sends[slot] != MPI_REQUEST_NULL) {
99                 MPI_Wait(&sends[slot], MPI_STATUS_IGNORE);
100             }
101             MPI_Isend(data + (size_t)chunk * chunk_size, chunk_size, MPI_INT,
102                     1, TAG_DATA, MPI_COMM_WORLD, &sends[slot]);
103         }
104         MPI_Waitall(BUFFER_COUNT, sends, MPI_STATUSES_IGNORE);
105
106         MPI_Request stop_request;
107         MPI_Isend(NULL, 0, MPI_INT, 1, TAG_STOP, MPI_COMM_WORLD, &stop_request);
108         MPI_Wait(&stop_request, MPI_STATUS_IGNORE);
109     } else {

```

WSL: Ubuntu Ln 94, Col 1 Spaces: 4 UTF-8 LF C

Figura 11. Productor con dos envíos activos y mensaje de parada enviado mediante `MPI_Isend`.

```

Labo3_paralela Visual Studio Code
pipeline_chunks.c x
111     const int initial = chunks < BUFFER_COUNT ? chunks : BUFFER_COUNT;
112     for (int slot = 0; slot < initial; ++slot) {
113         MPI_Irecv(buffers[slot], chunk_size, MPI_INT, 0, TAG_DATA,
114                 MPI_COMM_WORLD, &receives[slot]);
115     }
116
117     long long checksum = 0;
118     int posted = initial;
119     int processed = 0;
120     while (processed < chunks) {
121         int slot = MPI_UNDEFINED;
122         MPI_Waitany(BUFFER_COUNT, receives, &slot, MPI_STATUS_IGNORE);
123         if (slot == MPI_UNDEFINED) {
124             fprintf(stderr, "Error: no quedan recepciones activas.\n");
125             MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
126         }
127
128         long long partial = 0;
129         for (int i = 0; i < chunk_size; ++i) {
130             partial += buffers[slot][i];
131         }
132         checksum += partial;
133         ++processed;
134
135         if (detailed) {
136             printf("[rank 1] chunk %d/%d procesado, suma_parcial=%lld\n",
137                    processed, chunks, partial);
138             fflush(stdout);
139         }
140
141         if (posted < chunks) {
142             MPI_Irecv(buffers[slot], chunk_size, MPI_INT, 0, TAG_DATA,
143                     MPI_COMM_WORLD, &receives[slot]);
144             ++posted;
145         } else {
146             receives[slot] = MPI_REQUEST_NULL;
147         }
148     }

```

WSL: Ubuntu Ln 111, Col 1 Spaces: 4 UTF-8 LF C

Figura 12. Consumidor con dos buffers, `MPI_Irecv` y `MPI_Waitany`.

```

Labo3_paralela Visual Studio Code
TERMINAL
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
ihan@VICTUS:~/Labo3_paralela$ mpirun -np 2 ./build/pipeline_chunks 4096 1024 --detalle
[rank 1] chunk 1/4 procesado, suma_parcial=49776
[rank 1] chunk 2/4 procesado, suma_parcial=50928
[rank 1] chunk 3/4 procesado, suma_parcial=51504
[rank 1] chunk 4/4 procesado, suma_parcial=50352
PIPELINE_CHUNKS total_enteros=4096 chunk_enteros=1024 chunks=4 bytes=16384 tiempo_total_s=0.000046058
checksum=202560 stop_recibido=si
WSL: Ubuntu bash | UTF-8

```

Figura 13. Procesamiento de cuatro chunks y recepción correcta del mensaje de parada.

La prueba dividió 4,096 enteros en cuatro chunks, el rank 1 procesó los cuatro bloques, obtuvo un checksum de 202,560 y al final confirmó que también recibió el mensaje de parada.

Inciso 2 Mensaje de parada

Cuando ya terminó de mandar los datos, el rank 0 envía con **MPI_Isend** un mensaje que lleva el tag **TAG_STOP**, y el rank 1 lo recibe después de procesar todos los chunks para salir del ciclo. Se usó un tag distinto al de los datos para que el final del trabajo no se confunda con un bloque vacío.

Inciso 3 Tamaño del chunk y tiempo total

Chunk bytes	Promedio ms	Desviación ms
1,024	6.485545	0.204174
4,096	7.076982	0.398322
16,384	4.381741	0.151591
65,536	3.795057	0.214758
262,144	3.651354	0.125270
1,048,576	4.226186	0.187503

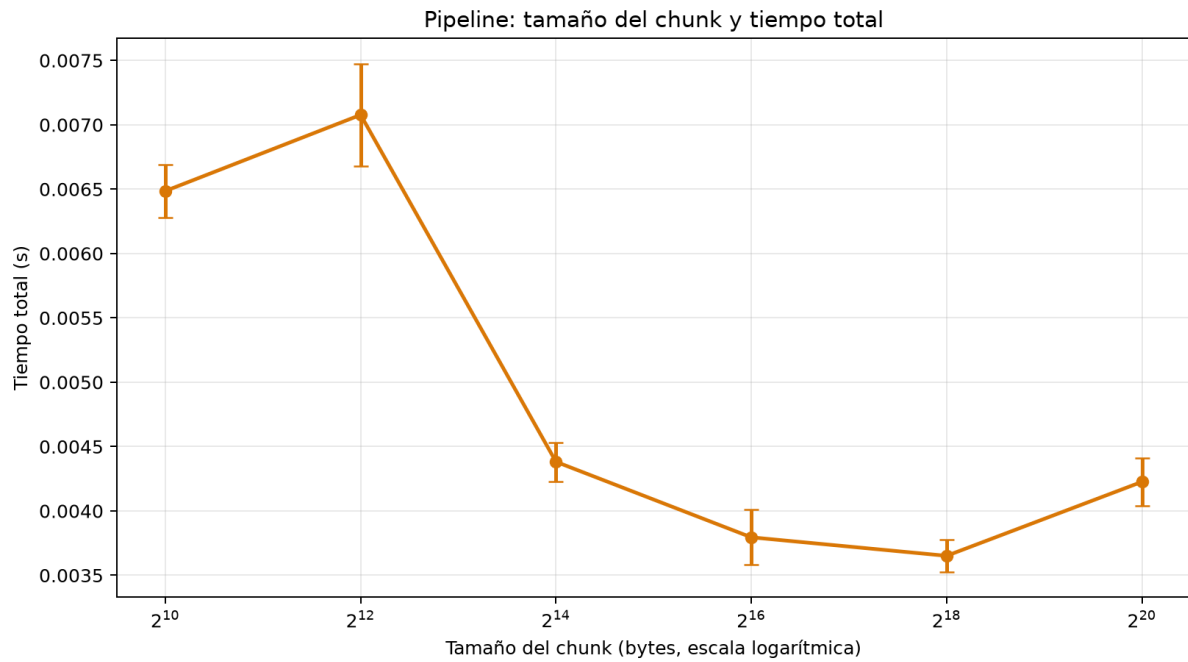


Figura 14. Tiempo total para 4,194,304 enteros según el tamaño del chunk.

Los chunks pequeños terminaron generando demasiados mensajes, por eso 1,024 y 4,096 bytes tardaron 6.485545 y 7.076982 ms. Al aumentar el bloque el tiempo bajó hasta 3.651354 ms con chunks de 262,144 bytes; al probar 1,048,576 bytes volvió a subir a 4.226186 ms, ya que quedaron solamente dieciséis chunks y hubo menos oportunidad de recibir un bloque mientras se procesaba el anterior. En esta computadora, 262,144 bytes dio el mejor resultado.